

Reporte de Actividad — Prolog

Práctica 4

Pablo Fernando Ruiz Perez 379207 Andres Mendez Huerta

Jose Carlos Gallegos Mariscal

22 de Mayo de 2026

Repositorio: <https://github.com/PabRuizzz/Paradigmas>

1. Introducción

En la siguiente práctica se tiene como objetivo introducir el paradigma lógico mediante el lenguaje de programación Prolog. A diferencia del paradigma funcional o imperativo, la programación lógica no se centra en como resolver un problema paso a paso mediante instrucciones secuenciales, sino en que se conoce del problema, delegando la resolución de la tarea a una máquina de inferencia automática.

Prolog se caracteriza por basarse en el cálculo de predicados de primer orden. Los programas en Prolog constan de una base de conocimientos conformada explícitamente por hechos y reglas. El entorno evalúa consultas cruzando estas definiciones de manera declarativa para encontrar respuestas válidas o comprobar la veracidad de una premisa mediante la unificación de términos y el mecanismo automático de backtracking.

2. Instalación y Verificación desde la Terminal (CMD)

Para trabajar con Prolog, la implementación más extendida y estándar en el entorno académico es SWI-Prolog. En sistemas operativos Windows, se puede realizar la instalación de manera ágil desde la consola de comandos (CMD) o PowerShell utilizando el gestor de paquetes nativo de Windows (winget), evitando así descargas o configuraciones manuales de variables de entorno.

Instalación mediante CMD:

```
winget install SWI-Prolog.SWI-Prolog
```

Verificación de la instalación:

Una vez concluido el proceso de instalación, se debe verificar que el entorno esté correctamente configurado y accesible de forma global. Para ello, se abre una nueva ventana de la terminal (CMD) e introduce el siguiente comando para consultar la versión actual del compilador:

```
swipl --version
```

Al ejecutar este comando, la terminal retornará un mensaje detallando la versión instalada, lo cual confirma que el compilador está listo para usarse.

Para iniciar el entorno interactivo de Prolog directamente desde la línea de comandos, basta con ingresar:

```
swipl
```

3. Sintaxis de Hechos

Un hecho en Prolog representa una afirmación incondicional que se asume siempre como verdadera dentro de nuestra base de conocimientos. Describe las propiedades inherentes de un objeto o las relaciones fijas que existen entre varios objetos.

Guía para escribir hechos:

- Los nombres de las propiedades, relaciones y objetos deben iniciar obligatoriamente con letra minúscula (los términos que inician con mayúscula son interpretados por el compilador como variables).
- El nombre de la relación aparece siempre como el primer término (denominado predicado).
- Los objetos involucrados aparecen como argumentos cerrados entre paréntesis y separados por comas.
- Un punto (.) debe terminar estrictamente cada hecho para indicar el fin de la cláusula.

Ejemplos de Hechos:

Propiedad Unaria: Establece una característica directa sobre un único objeto.

```
lazy(juan).
```

Declara formalmente el hecho de que Juan es perezoso. El predicado es "lazy" y el argumento es "juan".

Relación Binaria: Vincula dos objetos distintos bajo un mismo criterio conceptual.

```
loves_to_eat(jorge, pasta).
```

Declara el hecho de que "A Jorge le encanta comer pasta", estableciendo una relación de preferencia alimentaria donde el sujeto es jorge y el objeto asociado es pasta.

4. Sintaxis de Reglas

Una regla representa una afirmación condicional dentro de la base de conocimientos. Establece que una conclusión o consecuencia es verdadera si y solo si una o más condiciones especificadas en el cuerpo de la regla se cumplen de manera simultánea. Permite que el sistema infiera de forma lógica nueva información a partir de los hechos preexistentes.

Estructura Sintáctica:

```
cabeza :- cuerpo.
```

- Cabeza (Conclusión): Especifica el predicado o hecho que se quiere demostrar o deducir.
- `:-`: Se traduce e interpreta textualmente como "si" o "proviene de".
- Cuerpo (Condiciones): Consiste en una serie de hechos, reglas o metas secundarias que deben validarse. Si se separan por comas, representan una conjunción lógica. Si se separan por puntos y comas, representan una disyunción lógica.

Ejemplos de Reglas:

```
happy(lili) :- dances(lili).
```

Establece de manera condicional que "Lili se alegra si Lili baila". La cabeza `happy(lili)` se inferirá como verdadera únicamente si se logra comprobar el hecho `dances(lili)` en la base de datos de conocimiento.

Condición Compuesta con Conjunción Lógica

```
goToPlay(ryan) :- isClosed(school), free(ryan).
```

Determina la regla de que "Ryan irá a jugar si la escuela está cerrada Y además él se encuentra libre". Ambas premisas en el cuerpo, separadas por la coma, deben satisfacerse al mismo tiempo para que la conclusión general sea declarada verdadera por el motor de inferencia.

5. Relaciones en Prolog

En el diseño de programas en Prolog, el pilar fundamental consiste en especificar explícitamente las relaciones existentes entre los objetos y sus propiedades asociadas. Cuando el usuario realiza una consulta, el motor de ejecución busca correspondencias y patrones lógicos para validar si la relación planteada es verdadera o falsa.

Por ejemplo, si planteamos la afirmación cotidiana "José tiene una bicicleta", conceptualmente estamos declarando una relación de propiedad binaria que vincula formalmente al objeto `jose` con el objeto `bicicleta`.

Para comprender cómo se estructuran y expanden de forma avanzada las relaciones, observemos el siguiente fragmento adaptado de una base de conocimientos familiar.

```
% Hechos base de género
female(pam).
female(liz).
male(tom).
male(bob).

% Hechos base de parentesco (Relación raíz)
parent(pam, bob).
```

```
parent(tom, bob).  
parent(tom, liz).
```

A partir de estas relaciones directas, podemos estructurar reglas lógicas que definen de forma abstracta nuevas relaciones más complejas y refinadas sin necesidad de declararlas explícitamente como hechos estáticos:

Ejemplo de Relación Madre:

```
mother(X, Y) :- parent(X, Y), female(X).
```

Esta regla define la relación donde X es madre de Y. Para que el motor lo valide, primero debe verificarse la relación de parentesco base y simultáneamente comprobarse la propiedad biológica de que X es de género femenino.

Ejemplo de Relación Hermana (Con Control de Identidad):

```
sister(X, Y) :- parent(Z, X), parent(Z, Y), female(X), X \== Y.
```

Modela de forma detallada la relación de hermandad. Determina que X es hermana de Y si comparten exactamente el mismo progenitor Z, se comprueba que X es del género requerido, y se introduce de forma indispensable el operador de desigualdad para asegurar que una persona no sea evaluada ni devuelta erróneamente como hermana de sí misma al realizar consultas generales sobre la base de datos.

6. Ejercicios

Torres de Hanoi

Codigo:

```
1 % Caso base: Mover 1 disco del origen al destino
2 hanoi(1, Origen, Destino, _, _) :-
3     escribir_movimiento(Origen, Destino).
4
5 % Caso recursivo: Mover N discos
6 hanoi(N, Origen, Destino, Auxiliar, Contador) :-
7     N > 1,
8     M is N - 1,
9     % Paso 1: Mover N-1 discos del Origen al Auxiliar usando el Destino
10    hanoi(M, Origen, Auxiliar, Destino, Contador),
11    % Paso 2: Mover el disco grande restante al Destino
12    escribir_movimiento(Origen, Destino),
13    % Paso 3: Mover los N-1 discos del Auxiliar al Destino usando el Origen
14    hanoi(M, Auxiliar, Destino, Origen, Contador).
15
16 % Predicado auxiliar para imprimir el movimiento en pantalla
17 escribir_movimiento(Origen, Destino) :-
18     write('Mover disco desde la torre '), write(Origen),
19     write(' hasta la torre '), write(Destino), nl.
20
21 % Predicado principal para iniciar el juego de forma más cómoda
22 resolver_hanoi(N) :-
23     write('Pasos para resolver las Torres de Hanoi con '), write(N), write(' discos:'), nl,
24     hanoi(N, 'A', 'C', 'B', _).
```

Ejecucion:

```
1 ?- resolver_hanoi(3).
Pasos para resolver las Torres de Hanoi con 3 discos:
Mover disco desde la torre A hasta la torre C
Mover disco desde la torre A hasta la torre B
Mover disco desde la torre C hasta la torre B
Mover disco desde la torre A hasta la torre C
Mover disco desde la torre B hasta la torre A
Mover disco desde la torre B hasta la torre C
Mover disco desde la torre A hasta la torre C
true
```

Mono Y el Platanó

Codigo:

```

9 % Acción 1: Tomar la banana
10 % El mono puede tomar la banana si está sobre la caja en el centro y no la tiene aún.
11 mover(estado(centro, sobre_caja, centro, no),
12     tomar,
13     estado(centro, sobre_caja, centro, si)).
14
15 % Acción 2: Subirse a la caja
16 % El mono puede subir si está en el suelo en el mismo lugar que la caja.
17 mover(estado(Pos, suelo, Pos, Ban),
18     subir,
19     estado(Pos, sobre_caja, Pos, Ban)).
20
21 % Acción 3: Empujar la caja
22 % El mono puede empujar la caja de una Posición 1 a una Posición 2 si ambos están en la Posición 1 en el suelo.
23 mover(estado(Pos1, suelo, Pos1, Ban),
24     empujar(Pos1, Pos2),
25     estado(Pos2, suelo, Pos2, Ban)) :-
26     Pos1 \== Pos2.
27
28 % Acción 4: Caminar/Moverse solo
29 % El mono puede caminar de Pos1 a Pos2 si está en el suelo.
30 mover(estado(Pos1, suelo, PosCaja, Ban),
31     caminar(Pos1, Pos2),
32     estado(Pos2, suelo, PosCaja, Ban)) :-
33     Pos1 \== Pos2.
34
35 % Caso base: Si el estado actual ya muestra que el mono tiene la banana, el plan ha terminado.
36 solucion(estado(_, _, _, si), []).
37
38 % Caso recursivo: Encontrar una acción válida, ejecutarla y resolver el nuevo estado.
39 solucion(EstadoActual, [Accion | ListaAcciones]) :-
40     mover(EstadoActual, Accion, EstadoSiguiente),
41     solucion(EstadoSiguiente, ListaAcciones).

```

Ejecucion:

```

1 ?- solucion(estado(puerta, suelo, ventana, no), X).
X = [caminar(puerta, ventana), empujar(ventana, centro), subir, tomar]

```

7. Conclusión

A través del desarrollo de esta actividad, se comprendió de manera clara el funcionamiento del paradigma lógico por medio del lenguaje Prolog, contrastándolo directamente con enfoques previamente analizados como el paradigma funcional de Haskell. Se logró asimilar la transición de diseñar algoritmos basados en flujos secuenciales de control e instrucciones explícitas hacia un entorno completamente declarativo basado en la definición rigurosa de bases de conocimientos conformadas por hechos inmutables y reglas de inferencia.

El dominio de la sintaxis permitió reconocer el cuidado estricto que requiere Prolog en la distinción de identificadores, donde el uso preciso de minúsculas define átomos o constantes y las mayúsculas delimitan variables dinámicas. Estas competencias técnicas cimientan la base necesaria para la posterior implementación de técnicas complejas dentro del paradigma, tales como la recursión avanzada, el control de flujos con backtracking y la manipulación estructurada de listas.